



Editorial

Tasks, test data and solutions were prepared by: Fran Babić, Toni Brajko, Jakov Celin, Ivan Janjić, Josip Klepec and Martina Licul.

Implementation examples are given in attached source code files.

Task Osobna

Prepared by: Martina Licul

Necessary skills: strings, loops

In this task, we need to extract the name, surname, birthdate, and personal identification number from three given strings.

The name and surname are located in the third string. To extract them, we loop through the string until we encounter the character '<'. The first part of the string is the name, and the second part, after two consecutive '<' symbols, is the surname. The surname extends until the next '<' or the end of the string. After extracting both, we ensure that only the first letter of each is capitalized.

The birthdate is in the second string. It is represented by six digits: two for the year, two for the month, and two for the day. To determine the correct year, we check if the first two digits of the year are less than or equal to 24. If so, we assume the year is in the 2000s, otherwise it is in the 1900s.

The personal identification number is found in the first string, between the 16th and 26th positions. Extracting this substring gives the personal identification number.

Finally, the extracted data is printed in the format specified by the task.

Task Skokovi

Prepared by: Toni Brajko

Necessary skills: loops

We maintain an interval $[l, r]$ that represents the smallest and largest value that we can reach from the current positions. Initially, we set $l = r = a_0$, since we start at position a_0 .

For each position a_i (where $i > 0$), we check if a_i is within the interval extended by K , i.e., we check if:

$$l - K \leq a_i \leq r + K.$$

If this condition is satisfied, it means that we can reach position i . In that case, we update the interval $[l, r]$ to include the value a_i , that is:

$$l = \min(l, a_i),$$

$$r = \max(r, a_i).$$

If a_i is not within this interval, we cannot reach position i , and the interval remains unchanged.

The solution passes through the array once, so the time complexity is $O(N)$.

Example (from the problem):

For the array $a = [5, 4, 8, 7, 2]$ and $K = 2$, the intervals will be updated as follows:

- Initial interval: $[5, 5]$
- Position $a_1 = 4$: within the interval $[5 - 2, 5 + 2] = [3, 7]$, so we update the interval to $[4, 5]$
- Position $a_2 = 8$: outside the interval $[4 - 2, 5 + 2] = [2, 7]$, we cannot reach this position, the interval remains $[4, 5]$



- Position $a_3 = 7$: within the interval $[4 - 2, 5 + 2] = [2, 7]$, so we update the interval to $[4, 7]$
- Position $a_4 = 2$: within the interval $[4 - 2, 7 + 2] = [2, 9]$, so we update the interval to $[2, 7]$

In the end, we print which positions are reachable: positions 0, 1, 3, and 4 are reachable, while position 2 is not.

Task Učiteljica

Prepared by: Fran Babić

Necessary skills: graph theory, set

Let us note that, due to the conditions in the input data, the structure of relationships between employees can be imagined as a tree if the connections were undirected edges between the nodes (employees). However, due to the nature of this task, it makes more sense to view the given connections as directed edges. In this case, the question of whether the given relationship structure is a valid *hierarchy* becomes equivalent to the question of whether the imagined graph is a valid directed tree. A directed tree must have its *root*, and it must be possible to reach every other node from the root using the given directed connections.

For the first two subtasks, it was sufficient to iterate over all the nodes at the start and after each change, checking whether any of them is the root of the tree. The time complexity of such an algorithm is $O(N^2Q)$. For the third and fourth subtasks, it is enough to observe that in a directed tree only the root can (and must) have an in-degree of 0, i.e., there must be a unique node with no superior. This reduces the number of nodes we need to check, and we can improve the algorithm's complexity to $O(NQ)$ with a clever implementation.

Finally, for full points, it is necessary to observe that the graph under the task conditions is a directed tree if and only if there is a unique employee with no direct superiors (a node with an in-degree of zero), and all other employees have exactly one superior. Clearly, this is a necessary condition. The proof that this is also a sufficient condition is left as an exercise for the reader. The algorithm that solves this task can now be implemented with a complexity of $O((N + Q) \log N)$. You can find the details in the official source code.

Task Učiteljica

Prepared by: Jakov Celin

Necessary skills: sweep line, segment tree, inclusion-exclusion principle

Let's call an interval *good* if there exists a number that appears exactly once, a number that appears exactly twice, ..., and a number that appears exactly K times.

For the solution to the first subproblem, it was sufficient to fix each interval, add elements one by one, and then check whether the fixed interval is good. The complexity of the described algorithm is $O(N^2K)$.

In the second subproblem, the elements of the array are numbers between 1 and K . We observe that an interval is good if every number from 1 to K appears a different number of times and if the interval is of length $\frac{K(K+1)}{2}$. What remains is to check $O(N)$ intervals using the method from the first subproblem.

In the third subproblem, we want to count intervals where there exists an element that appears exactly once. For each index i between 1 and N , we want to describe using inequalities when the element at position i appears only once.

- Find the largest index L_i smaller than i such that $a[L_i] = a[i]$ and the smallest index R_i larger than i such that $a[R_i] = a[i]$.
- We observe that the element i will appear in the interval of indices $[x, y]$ exactly once if and only if $L_i < x \leq i \leq y < R_i$.



The subproblem is now reduced to counting pairs of indices x and y such that there exists at least one index i for which $L_i < x \leq i \leq y < R_i$. The solution to this problem is found by considering the conditions for each index as rectangles in the plane. Then, we observe that the number of valid intervals is the number of integer points covered by at least one rectangle, i.e., the number of integer points covered by the union of all rectangles. The problem of finding the area of the union of all rectangles is well known, and one explanation of the algorithm for finding the union area can be found at [this link](#). The algorithm's complexity is $O(N \log N)$.

In the fourth subproblem, we will use the properties from the third subproblem, and for each number x between 1 and K , we will find rectangles such that for each index i from 1 to N , the element a_i appears exactly x times. We observe that the solution to the problem is the area of the points covered by at least one rectangle for each x . If we represent all the points covered by the rectangles for a fixed x as the set S_x , we are essentially looking for the number of elements in the set $S_1 \cap S_2 \cap \dots \cap S_K$. Directly computing the intersection of sets is extremely difficult, but using the inclusion-exclusion principle, we can calculate the solution using unions of sets. Let S be the set containing $S_1, S_2, \dots, S_K$, i.e., $S = S_1, S_2, \dots, S_K$. According to the inclusion-exclusion principle, $S_1 \cap S_2 \cap \dots \cap S_K$ is given by

$$\sum_{Q \subseteq S} f(Q) \cdot (-1)^{|Q|}$$

where $f(Q)$ is the area of the union of all rectangles in the set Q . The algorithm's complexity is $O(2^{K-1} K N \log N)$. For implementation details, refer to the provided source code.

Task Zbunjenost

Prepared by: Ivan Janjić

Necessary skills: dynamic programming, graph theory

Preliminary observation is that a simple closed loop (cycle onwards) is a polygon whose vertices are a subset of the original vertices where order was preserved. For the first subtask it was enough to check every subset if it is a cycle. Complexity is $O(n2^n)$ even though constraints allow for somewhat slower algorithms.

Looking at one cycle one can notice that it is also triangulated and that those triangles form a subset of triangles that original polygon was decomposed into. That motivates the introduction of the following graph: vertices represent triangles of the original triangulation, and two vertices are connected by an edge if they share a side. Two crucial observations: the graph is a tree and every connected component corresponds to a cycle. Sketch of proof: in a triangulation there is always an "ear", a triangle whose two sides are edges of polygon, it is connected to one other triangle, remove it from the triangulation and inductively proceed on the rest. Connectedness is obviously necessary, and it can be shown that every connected component is a valid cycle.

Task now splits in two. First part is building the tree, and the second is counting the number of connected components. Tree can be naively built in $O(n^3)$ or $O(n^2)$ by finding all triangles and for each pair checking if they are connected. That was enough for the second and third subtasks respectively. For a full solution one had to come up with an $O(n)$ or $O(n \log n)$ algorithm, and here we present one of many: sort diagonals by length. It can be shown that the current diagonal is a part of an "ear" if we remove those "ears" in the given order. We keep vertices in a doubly linked list so removing an "ear" is removing a vertex not a part of the diagonal by updating the list. For every diagonal we keep which triangles contain it and connect them. For details of implementation see attached solution.

For counting connected components, root the tree in some vertex and calculate $dp(x)$ - number of connected components containing x and possibly some other vertices from its subtree. Transition looks like this:

$$dp(x) = \prod_{y \text{ child of } x} (dp(y) + 1)$$

From a subtree of a child one can take nothing or take a connected component containing that child,



exactly the definition of dp . Children are independent of each other so we just multiply all the values.

Final answer is $\sum_{x=1}^n dp(x)$ because every connected component has a unique vertex such that every other vertex is in its subtree.